

ECS
- double _time - std::size_t __livingEntities - std::queue< Entity > __freeEntities - std::unordered_map< Entity, Signature > __signatures - std::vector< void * > __componentStorages - std::vector< void * > __componentStorage
+ ECS() + ~ECS()=default + Entity createEntity() + void killEntity(Entity e) + Signature getSignature(Entity e) const + void setSignature(Entity e, Signature sig) + Entity addComponent(Entity e, const Component Type &component) + bool hasComponent(Entity e) const + const ComponentType *getConstComponent(Entity e) const + ComponentType *getComponent(Entity e) and 17 more... - void verifyComponentStorage(ComponentTypeID TypeID) - void verifyComponentStorage(ComponentTypeID TypeID)

ResourceManager
- std::unordered_map< std::string, sf::Texture > _textures - std::unordered_map< std::string, sf::Font > _fonts
+ bool loadTexture(const std::string &id, const std::string &filepath) + sf::Texture &getTexture(const std::string &id) + bool loadFont(const std::string &id, const std::string &filepath) + const sf::Font &getFont(const std::string &id) const + static ResourceManager &getInstance() - ResourceManager()=default - ~ResourceManager()=default - ResourceManager(const ResourceManager &)=delete - ResourceManager & operator =(const ResourceManager &)=delete

ISystem
+ ISystem(ECS &ecs) + virtual ~ISystem()=default + virtual void update(double dt)=0 + ISystem(ECS &ecs) + virtual ~ISystem()=default + virtual void update(double dt)=0

Timer
- sf::Clock _clock - sf::Time _lastTick - uint16_t _frameNumber
+ Timer() + ~Timer() + sf::Int64 getLastTick() + uint16_t getCurrentFrame() + void updateClock()

SafeQueue< InputState >
- std::queue< InputState > queue_ - std::mutex mutex_
+ SafeQueue()=default + void push(const InputState &value) + bool tryPop(InputState &value)

SafeQueue< std::vector< EntityUpdate > >
- std::queue< std::vector< EntityUpdate > > queue_ - std::mutex mutex_
+ SafeQueue()=default + void push(const std::vector< EntityUpdate > &value) + bool tryPop(std::vector< EntityUpdate > &value)

InputSystem
- std::map< sf::Keyboard::Key, GameAction > m_keyMappings - std::map< unsigned int, GameAction > m_joystickButtonMappings - std::map< GameAction, bool > m_currentActionStates - std::map< GameAction, bool > m_previousActionStates - unsigned int m_joystickId - float m_deadzone - float m_playerSpeed
+ InputSystem(ECS &ecs) + ~InputSystem()=default + void update(double dt) override + bool isActionActive(GameAction action) const + bool wasActionPressed(GameAction action) const + bool wasActionReleased(GameAction action) const + void setKeyMapping(sf::Keyboard::Key key, GameAction action) + void removeKeyMapping(sf::Keyboard::Key key) + void clearKeyMappings() + sf::Keyboard::Key getKeyForAction(GameAction action) const and 6 more... - void setDefaultMappings() - void updateJoystickInput() - void applyInputToPlayers()

RenderSystem
- sf::RenderWindow &_window
+ RenderSystem(ECS &ecs, sf::RenderWindow &window, ResourceManager &resourceManager) + void update(double dt) override

MoveSystem
+ MoveSystem(ECS &ecs) + ~MoveSystem() override =default + void update(double dt) override

UdpClient
- sf::UdpSocket m_socket - sf::IpAddress m_serverIp - unsigned short m_serverPort - std::atomic< bool > m_running - std::thread m_sendThread - std::thread m_rcvThread
+ UdpClient(sf::IpAddress serverIp="127.0.0.1", unsigned short serverPort=SERVER_PORT) + ~UdpClient() + bool start(unsigned short localPort=0) + void stop() + void join() - void sendThread() - void receiveThread()

Client
+ bool _running - std::unordered_map< std::size_t, std::size_t > serverToClientEntityRelation - float _shootCooldown - Entity _localPlayerEntity - sf::RenderWindow _window
+ Client(sf::IpAddress serverIp="127.0.0.1") + ~Client() + void update() + void render() + void processInput() - void applyUpdate(EntityUpdate &update)

SafeQueue< T >
- std::queue< T > queue_ - std::mutex mutex_
+ SafeQueue()=default + void push(const T &value) + bool tryPop(T &value)

